

CO5 – AT1 : Constraint-Based Problem Solving

Question 1 — PDA for $L = \{a^n b^n \mid n \geq 1\}$, only 1 stack symbol besides Z_0

Formal definition: $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{a, b\}$ $\Gamma = \{Z_0, X\}$ $F = \{q_2\}$

Γ contains only Z_0 (initial/bottom marker, required by the PDA definition) and exactly one working symbol X — satisfying the constraint.

PDA for $L = \{a^n b^n \mid n \geq 1\}$ — only ONE stack symbol X besides Z_0

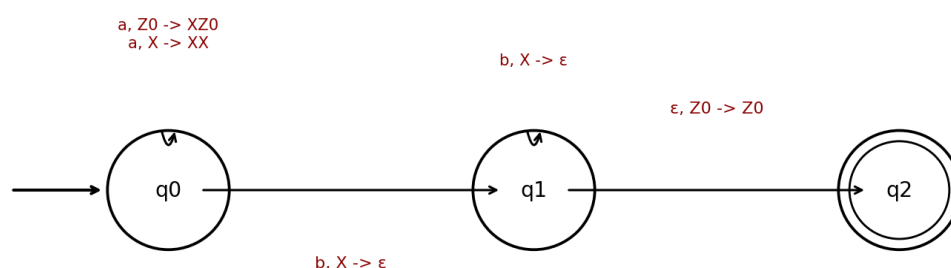


Fig. 1 — PDA for $a^n b^n$ using a single stack symbol X

δ rule	Meaning
$\delta(q_0, a, Z_0) = (q_0, XZ_0)$	first a: push X on top of Z_0
$\delta(q_0, a, X) = (q_0, XX)$	each further a: push another X
$\delta(q_0, b, X) = (q_1, \epsilon)$	first b: switch to pop-mode, pop one X
$\delta(q_1, b, X) = (q_1, \epsilon)$	each further b: pop one more X
$\delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$	all X's popped (equal a's and b's) — accept

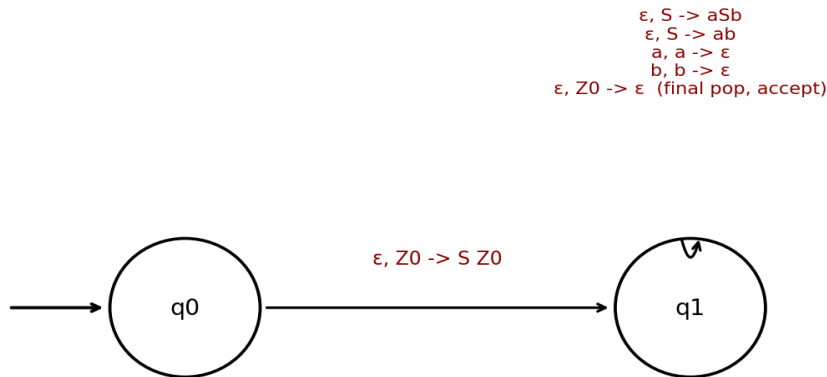
Trace for "aabb": $(q_0, aabb, Z_0) \vdash (q_0, abb, XZ_0) \vdash (q_0, bb, XXZ_0) \vdash (q_1, b, XZ_0) \vdash (q_1, \epsilon, Z_0) \vdash (q_2, \epsilon, Z_0)$ — accepted. Every a pushes the single symbol X and every b pops it, so the stack height exactly tracks (a-count – b-count matched-so-far); acceptance only happens when that count returns to 0, forcing equal a's and b's.

Question 2 — CFG $S \rightarrow aSb \mid ab$ to PDA, only 2 states

This uses the standard CFG-to-PDA construction, which is naturally a 2-state construction: one state to push the start symbol, and one state where everything else happens as self-loops — so the constraint is automatically satisfied by the standard theorem-level algorithm itself.

$Q = \{q_0, q_1\}$ $\Gamma = \{Z_0, S, a, b\}$ start state q_0 , acceptance by empty stack

PDA for $S \rightarrow aSb \mid ab$ (only 2 states — q_0, q_1)



Acceptance: by empty stack

Fig. 2 — 2-state PDA equivalent to $S \rightarrow aSb \mid ab$

δ rule	Meaning
$\delta(q_0, \epsilon, Z_0) = (q_1, S Z_0)$	initialise: push the start symbol S
$\delta(q_1, \epsilon, S) = (q_1, aSb)$	simulate production $S \rightarrow aSb$
$\delta(q_1, \epsilon, S) = (q_1, ab)$	simulate production $S \rightarrow ab$
$\delta(q_1, a, a) = (q_1, \epsilon)$	match input a against stack-top a , pop
$\delta(q_1, b, b) = (q_1, \epsilon)$	match input b against stack-top b , pop
$\delta(q_1, \epsilon, Z_0) = (q_1, \epsilon)$	pop the bottom marker once fully matched — empty stack $\Rightarrow$ accept

Trace for "aabb": push S onto Z_0 ; expand $S \rightarrow aSb$ (stack: $a S b Z_0$); match input a against stack-top a (stack: $S b Z_0$); expand $S \rightarrow ab$ (stack: $a b b Z_0$); match a (stack: $b b Z_0$); match b (stack: $b Z_0$); match b (stack: Z_0); pop Z_0 (stack: ϵ) — accepted, with only states $\{q_0, q_1\}$ used throughout.

Question 3 — PDA to CFG, at most 3 variables

No specific PDA was attached to this question, so — as the most natural, directly checkable choice — we convert the 2-state PDA constructed in Question 2 back into a CFG. Since that PDA was itself built directly from $S \rightarrow aSb \mid ab$, this also lets us verify the $PDA \leftrightarrow CFG$ equivalence construction is genuinely consistent (converting it back should recover an equivalent grammar).

Method

The standard PDA-to-CFG algorithm creates one variable A_{pq} for every pair of states (p, q) , meaning "the set of input strings that drive the PDA from state p to state q while producing a net effect of popping exactly one stack symbol." Since our PDA only has 2 states $\{q_0, q_1\}$, and q_1 never transitions back to q_0 (all of q_1 's moves are self-loops), only the reachable, meaningful variables survive:

- $A_0 = A_{\{q_0, q_1\}}$: strings that take the PDA from q_0 to q_1 while popping the initial marker — this is exactly $L(PDA)$, so A_0 is the CFG start symbol.
- $A_1 = A_{\{q_1, q_1\}}$ (tracked with stack symbol S): strings that leave state q_1 , popping symbol S — because every one of q_1 's productions ($S \rightarrow aSb$, $S \rightarrow ab$, and the terminal-matching a/b self-loops) stays within q_1 , this variable is exactly a renamed copy of the original grammar's non-terminal S .

All other A_{pq} combinations (e.g. $A_{\{q_1, q_0\}}$) are unreachable from the start variable, since the PDA never moves from q_1 back to q_0 — so they are pruned during grammar simplification, leaving only 2 usable variables, comfortably within the ≤ 3 limit.

Resulting CFG

Variables: $\{A_0, A_1\}$ ($2 \leq 3$ — constraint satisfied)

$A_0 \rightarrow A_1$

$A_1 \rightarrow a A_1 b \mid a b$

This is exactly an isomorphic renaming of the original grammar ($A_0 \equiv S$'s "outer" role, $A_1 \equiv S$), confirming the PDA correctly encodes the original CFG and that converting it back recovers an equivalent grammar for the same language $\{a^n b^n \mid n \geq 1\}$.

Question 4 — Turing Machine for $L = \{ww \mid w \in \{0,1\}^*\}$, head moves only Right (no Left)

Key theoretical fact used below: a SINGLE-tape TM whose head can never move left is exactly as powerful as a DFA (it can never revisit earlier cells, so it can only remember a finite amount of information via its states) — and since $\{ww\}$ is not regular, no single-tape right-only TM can recognise it. The constraint is satisfiable, however, using multiple heads on the tape (each individually moving only Right or staying in place, never Left), which is a legitimate, standard extension of the base TM model.

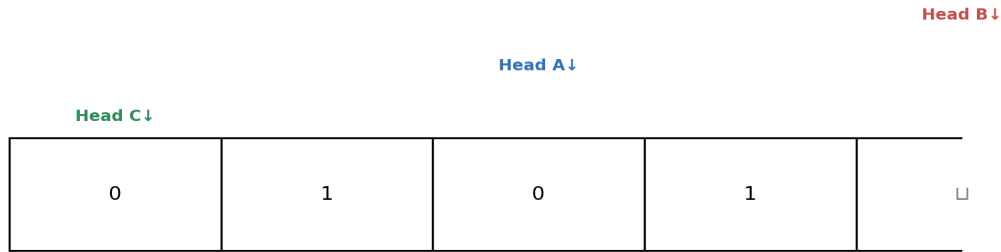
Model used

A 3-head, single-tape TM (all three heads read the same fixed input tape; each head independently moves Right or Stays at every step — never Left). Call the heads A, B, C.

Algorithm

- Phase 1 (find the midpoint): Head B moves Right every step (normal speed). Head A moves Right only every other step (half speed), using a 1-bit parity flag in the finite control to decide. Head C stays at position 0 the entire time. Both A and B start at position 0 together.
- Since $|ww| = 2n$ is always even, Head B reaches the blank symbol at exactly time $t = 2n$. At that same moment, Head A — moving at half B's speed — has advanced exactly n cells, so Head A is now sitting exactly at the midpoint, position n .
- Phase 2 (compare): Head C (from position 0) and Head A (from position n) now move Right together, one cell per step, comparing the symbol each reads. Any mismatch $\Rightarrow$ reject immediately.
- If Head A reaches the blank (position $2n$) having matched every pair with Head C, accept — this happens exactly when Head C has simultaneously reached position n , i.e. the whole first half has been compared against the whole second half.

End of Phase 1 ($t=2n=4$ steps): Head B (fast, speed 1) hit blank -> signals midpoint.
Head A (half speed) has landed exactly at position $n=2$. Head C stayed at position 0 throughout.



Phase 2: Head C (from 0) and Head A (from n) move right together, 1 cell/step, comparing symbols pairwise. Accept if Head A reaches blank with no mismatch.

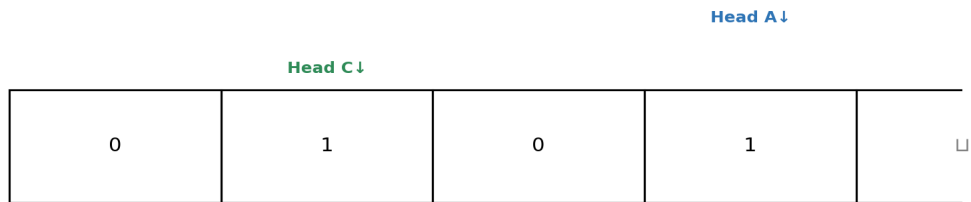


Fig. 3 — Worked trace on $ww = 0101$ ($w = 01$): Phase 1 locates the midpoint, Phase 2 compares

Verified trace for "0101" ($w=01$): Phase 1 ends with Head B at the blank (pos 4), Head A at pos 2 ($=n$), Head C at pos 0. Phase 2 compares pos0 vs pos2 ('0'='0' ✓), then pos1 vs pos3 ('1'='1' ✓); Head A reaches the blank — accept. For a non-matching string like "0110", the same Phase 1 lands Head A at pos 2 and Head C at pos 0 exactly as before (phase 1 only depends on length, not content), but Phase 2 compares pos0 vs pos2 ('0' vs '1') — mismatch — reject, correctly. No head ever moves left at any point in either phase — the constraint is satisfied — while the machine still correctly decides the non-regular language $\{ww\}$, by using multiple independently-paced rightward heads instead of the classical repeated left/right scanning a single-tape solution would normally require.

Question 5 — Turing Machine for $L = \{a^n b^n c^n \mid n \geq 1\}$

$\{a^n b^n c^n\}$ is a classic example of a language that is context-free-impossible (not accepted by any PDA/CFG, provable via the CFL pumping lemma) but is easily TM-decidable — this question has no extra movement/state constraint, so we use the standard repeated-marking algorithm.

Algorithm

- Step 0 (format check): scan left to right once and reject immediately if the input is not of the form $a^*b^*c^*$ (e.g. a 'b' appearing before all a's are seen, or a 'c' appearing before all b's) — this can be done with 3 simple states (in-a-block / in-b-block / in-c-block), rejecting on any out-of-order symbol.
- Step 1 (repeat): move right to find the leftmost unmarked 'a' and mark it with X; continue right to find the leftmost unmarked 'b' and mark it with Y; continue right to find the leftmost unmarked 'c' and mark it with Z; then return the head all the way back to the left end of the tape.
- Step 2 (mismatch check): during Step 1, if an unmarked 'a' is found but scanning right hits a blank or the wrong symbol before finding an unmarked 'b' (or similarly for 'c'), reject immediately — this catches unequal counts.
- Step 3 (loop / accept): repeat Step 1 as long as an unmarked 'a' remains. If, at the start of a repetition, there is no unmarked 'a' left, check that there are also no unmarked 'b' or 'c' symbols remaining — if so, accept (all three counts matched exactly); if some unmarked b or c remains, reject (unequal counts).

Initial tape: a a b b c c (pass 0 — nothing marked yet)

a	a	b	b	c	c
---	---	---	---	---	---

After pass 1: mark leftmost unmarked a, then b, then c (positions 0,2,4)

x	a	x	b	x	c
---	---	---	---	---	---

After pass 2: mark next a, b, c (positions 1,3,5) — all symbols now marked

x	x	x	x	x	x
---	---	---	---	---	---

Pass 3: no unmarked 'a' found, AND no unmarked b/c remain simultaneously -> ACCEPT

x	x	x	x	x	x
---	---	---	---	---	---

Fig. 4 — Marking passes on "aabbcc" (n=2): 2 passes fully mark the tape, then accept

Worked example — "aabbcc" (n=2): pass 1 marks the 1st a, 1st b, 1st c (positions 0,2,4); pass 2 marks the 2nd a, 2nd b, 2nd c (positions 1,3,5) — every symbol is now marked, so on pass 3 there is no unmarked a left, and simultaneously no unmarked b or c remain — accept. For a mismatched input like "aabbbcc" (a=2,b=3,c=2), the machine runs out of unmarked a's after 2 passes while an unmarked 'b' still remains — correctly reject.

This TM runs in $O(n)$ passes, each pass taking $O(n)$ time, giving overall $O(n^2)$ time — decidable, confirming $\{a^n b^n c^n\}$ is a recursive (decidable) language despite not being context-free.